Step 3 — Multiple MCP Servers at Once, With Tool Namespacing — Solution

> Back to the step · Hint 1 · Hint 2

Story: a real agent needs tools from more than one source — a notes server *and* a filesystem server. Two servers can expose the same tool name, so each discovered tool is namespaced `servername__toolname`, and a routing table maps that back to the real server + tool. The pool already did this in Step 1; this step is where it earns its keep. **If not:** two `search` tools would collide, and a call meant for one server would silently hit the other.

The pool code doesn't change this step — it already namespaces and routes. What's new is a **second server** and a task that needs both.

`servers/files_server.py`

Story: a second tiny server, a different domain. Now there are two independent tool sources, exactly like two teams' services. **If not:** you could never prove routing actually picks the right server — one server can't collide with itself.

```
# A second minimal MCP server (files), launched as its own subprocess.
# Independent of the notes server — a different domain, its own tools.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("files-server")

FILES = {
    "project": ["app.py", "config.yaml", "deploy.sh", "README.md"],
}


@mcp.tool()
def list_files(directory: str) -> str:
    """List the file names in a directory."""
    if directory not in FILES:
        return f"Error: no directory named {directory}"
    return "\n".join(FILES[directory])
```

```
@mcp.tool()
def read_file(path: str) -> str:
    """Read the text of one file by its path."""
    return f"(pretend contents of {path})"


if __name__ == "__main__":
    print("files server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

`src/servers_config.py` — now two servers

```
# The servers to connect to, as plain data. The agent never hardcodes
# these servers' tools — only how to launch them.
SERVERS = [
    {"name": "notes", "command": "python",
     "args": ["servers/notes_server.py"]},
    {"name": "filesystem", "command": "python",
     "args": ["servers/files_server.py"]},
]
```

Why the namespacing matters — and the gotcha

The pool namespaces every tool as `servername__toolname` and keeps a routing dict:

```
# built inside connect_all(), per discovered tool:
namespaced = name + NAME_SEPARATOR + tool.name     # "notes__search_notes"
self.routing[namespaced] = (name, tool.name)   # -> ("notes","search_notes")
```

The gotcha, worth stating plainly: OpenAI's function-name field allows only letters, digits, underscores, and hyphens. A human-readable `notes.search_notes` with a `.` is **rejected** by the API. That's why the separator is `__` (`NAME_SEPARATOR = "__"`), not `.`. The model only ever sees the namespaced name; `pool.call_tool` uses the routing dict to turn it back into the real `(server, tool)` pair before dispatching.

`main.py` — a task that needs both servers

```
# Step 3 demo: connect to BOTH servers and run a task that needs tools
# from each — printing which namespaced tool handled each step.
```

```
import asyncio
import sys

from src.agent import run_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import create_client
from src.server_pool import MCPServerPool
from src.servers_config import SERVERS

TASK = ("Check my notes for anything about the deploy process, and also "
        "list the files in the project folder.")


async def run_demo():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    pool = MCPServerPool(SERVERS, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    print(f"Connected servers: {list(pool.sessions.keys())}")
    print(f"Tools discovered: {len(pool.named_tools)}")
    try:
        print(f"\nTask: {TASK}")
        result = await run_agent(
            TASK, pool, client, max_iterations=config.max_iterations
        )
        for step in result.steps:
            print(f"  -> {step['tool']} {step['args']}")
        print(f"Answer: {result.answer}")
    finally:
        await pool.aclose()


def main():
    asyncio.run(run_demo())


if __name__ == "__main__":
    main()
```

Expected output (`python main.py`) — one call routes to the notes server, one to the filesystem server:

```
Connected servers: ['notes', 'filesystem']
Tools discovered: 4

Task: Check my notes for anything about the deploy process, and also list
the files in the project folder.
  -> notes__search_notes {'query': 'deploy'}
  -> filesystem__list_files {'directory': 'project'}
Answer: Your notes say the deploy process runs deploy.sh after the staging
check; the project folder contains app.py, config.yaml, deploy.sh and
README.md.
```

`tests/integration/test_agent.py` — add case 3

Add the `FILES` server and this case to the integration test:

```
FILES = {"name": "filesystem", "command": "python",
         "args": ["servers/files_server.py"]}


async def test_routing_across_servers() -> None:
    config, client, pool = await connect([NOTES, FILES])
    task = ("Check my notes for anything about the deploy process, and also "
            "list the files in the project folder.")
    out = await run_agent(task, pool, client, config.max_iterations)
    tools_used = []
    for step in out.steps:
        tools_used.append(step["tool"])
    result = "OK"
    if "notes__search_notes" not in tools_used:
        result = "WRONG"
    if "filesystem__list_files" not in tools_used:
        result = "WRONG"
    await pool.aclose()
    print(f"case 3 (routes to both servers)        -> {result}")
```

and call it from `main()`:

```
    await test_routing_across_servers()
```

Expected output:

```
case 1 (tools discovered + namespaced) -> OK
case 2 (chained search then read)      -> OK
```

```
case 3 (routes to both servers)        -> OK
```

The case checks the *tools used*, not just the final text — that's what proves each namespaced call reached the server you expected, not that the answer happened to look right.